

Paradigmi di Programmazione - A.A. 2023-24

Esame Scritto del 09/07/2024

CRITERI DI VALUTAZIONE:

La prova è superata se si ottengono almeno 12 punti negli esercizi 1,2,3 e almeno 18 punti complessivamente.

Esercizio 1 [Punti 4]

Applicare la β -riduzione **con strategia call-by-name** alla seguente λ -espressione fino a raggiungere una espressione non ulteriormente riducibile o ad accorgersi che la derivazione è infinita:

$$(\lambda y.y)((\lambda x.\lambda y.\lambda z.xyz)z)x)$$

Nella soluzione, mostrare tutti i passi di riduzione calcolati, sottolineando ad ogni passo la porzione di espressione a cui si applica la β -riduzione (redex) ed evidenziando le eventuali α -conversioni.

SOLUZIONE:

Una possibile soluzione:

$$\begin{aligned} & (\lambda y.y)((\lambda x.\lambda y.\lambda z.xyz)z)x) \\ \rightarrow & \underline{((\lambda x.\lambda y.\lambda z.xyz)z)x)} \\ \equiv_{\alpha} & \underline{((\lambda x.\lambda y.\lambda k.xyz)z)x)} \\ \rightarrow & \underline{((\lambda y.\lambda k.zyk)x)} \\ \rightarrow & (\lambda k.zxk) \end{aligned}$$

Esercizio 2 [Punti 4]

Indicare il tipo della seguente funzione OCaml, mostrando i passi fatti per inferirlo:

```
fun x -> fun y -> match x with
  | (1,f) -> true
  | (n,f) -> (f y)
```

SOLUZIONE:

Struttura del tipo:

$X \rightarrow Y \rightarrow \text{RIS}$

Uso per convenzione X come variabile di tipo per la variabile x , RIS come variabile di tipo del risultato, e $A, B, C, \dots$ come variabili di tipo "fresche" per la definizione dei vincoli.

Vincoli:

$X = \text{int} * F$ (da pattern)
 $\text{RIS} = \text{bool}$ (da primo caso del p.m.)
 $F = Y \rightarrow \text{RIS}$ (da secondo caso del p.m.)

Ne consegue:

$X = \text{int} * (Y \rightarrow \text{bool})$
 $Y = Y$
 $\text{RIS} = \text{bool}$

Tipo inferito:

$(\text{int} * (Y \rightarrow \text{bool})) \rightarrow Y \rightarrow \text{bool}$

che in sintassi OCaml corrisponde a:

$(\text{int} * ('a \rightarrow \text{bool})) \rightarrow 'a \rightarrow \text{bool}$

Esercizio 3 [Punti 7]

Definire in OCaml tre funzioni `pairs`, `split` e `paridispari` con tipi

```
pairs : 'a list -> 'a * 'a list
split : 'a * 'b list -> 'a list * 'b list
paridispari : int list -> int list * int list
```

La funzione `pairs` prende una lista $[e_1; e_2; e_3; \dots; e_k]$ di tipo `'a list` e restituisce una lista in cui gli elementi vengono messi in coppia a due a due (il primo con il secondo, il terzo con il quarto, ecc.), come nel seguente esempio:

```
pairs [1;2;3;4;5;6] = [(1, 2); (3, 4); (5, 6)]
```

se la lista passata ha lunghezza dispari, l'ultimo elemento viene ignorato, come nel seguente esempio:

```
pairs [1;2;3;4;5;6;7] = [(1, 2); (3, 4); (5, 6)]
```

La funzione `split` prende una lista di coppie $[(a_0, b_0); (a_1, b_1); \dots; (a_k, b_k)]$ di tipo `'a * 'b list` e restituisce una coppia di liste contenenti rispettivamente tutti i primi e tutti i secondi elementi delle singole liste, come nel seguente esempio:

```
split [(0,5);(1,6);(2,7)] = ([0; 1; 2], [5; 6; 7])
```

NOTA: non utilizzare la funzione di libreria `List.split` per risolvere questo esercizio.

La funzione `paridispari` prende una lista di interi `[n0;n1;n2;...;nk]` di tipo `int list` e restituisce una coppia di liste `(lis1,lis2)` di tipo `int list * int list` in cui `lis1` contiene gli elementi in posizioni pari nella lista di partenza (partendo a contare da 0, ossia `[n0;n2;...]`) mentre `lis2` contiene gli elementi in posizioni dispari (ossia `[n1;n3;...]`). Ad esempio:

```
paridispari [4;1;5;7;9;3;2] = ([4;5;9;2],[1;7;3])
```

SOLUZIONE:

Una possibile soluzione:

```
let rec pairs lis =
  match lis with
  | [] -> []
  | x::[] -> []
  | x::y::lis' -> (x,y)::(pairs lis')

let rec split lis =
  match lis with
  | [] -> ([],[])
  | (x,y)::lis' -> let (l1,l2) = split lis' in (x::l1,y::l2);;

let paridispari lis =
  match list with
  | [] -> ([],[])
  | x::[] -> ([x],[])
  | x::y::lis' -> let (l1,l2) = paridispari lis' in (x::l1,y::l2);;
```

Esercizio 4 [Punti 15]

Si estenda il linguaggio MiniCaml visto a lezione con un nuovo tipo di dato `IntMultSet` che permette di dichiarare multi-insiemi di interi (insiemi di interi che possono contenere lo stesso numero anche più di una volta), con le seguenti operazioni:

```
Empty                /* Costruisce un multi-insieme vuoto */
Add(n, ms)           /* Costruisce un multi-insieme ottenuto aggiungendo un
                      intero descritto da n al multi-insieme descritto da ms */
Remove(n, ms)        /* Costruisce un multi-insieme ottenuto rimuovendo tutte
                      le occorrenze dell'intero descritto da n dal
                      multi-insieme descritto da ms */
HowMany(pred, ms)    /* Calcola il numero di elementi di ms che soddisfano il
                      predicato (funzione da interi a booleani) descritto da pred. */
```

Esempi (in sintassi concreta):

```
let ms1 = Add(2+2,Add(1,Empty));;      /* ms1 contiene [1;4] */
let ms2 = Add(3,Add(4,Add(3,ms1)));;   /* ms2 contiene [1;4;3;4;3] */
let ms3 = Remove(3,ms2);;              /* ms3 contiene [1;4;4] */
HowMany((fun x -> x>2), ms3);;         /* risultato 2 (ms3 contiene due valori >2) */
HowMany((fun x -> x>5), ms3);;         /* risultato 0 (ms3 non contiene valori >5) */
```

Si modifichi l'implementazione dell'interprete del linguaggio con quanto serve per gestire i costrutti per operare su collezioni di interi descritte in precedenza.

SOLUZIONE:

Una possibile soluzione:

```
type exp =
...
| Empty
| Add of exp * exp
| Remove of exp * exp
| HowMany of exp * exp

type evT = ... | MSet of evT list

let rec eval e s =
  match e with
  ...
  | Empty -> MSet []
  | Add (e1,e2) ->
    let elem = eval e1 s in
    let coll = eval e2 s in
    ( match (elem, coll) with
      | (Int n, MSet lst) -> MSet (elem::lst)
      | _ -> failwith "type error"
    )
  | Remove (e1,e2) ->
    let elem = eval e1 s in
    let coll = eval e2 s in
    ( match (elem, coll) with
      | (Int n, MSet lst) -> MSet (List.filter (fun x -> (x<>elem)) lst)
      | _ -> failwith "type error"
    )

  | HowMany (e1,e2) ->
    let pred = eval e1 amb in
    let coll = eval e2 amb in
    ( match(pred, coll) with
      | (Closure(arg, body, fdeclenv), MSet(lst)) ->
        let check acc x =
          let newenv = bind fdeclenv arg x in
          let result = eval body newenv in
          match result with
          (
            | Bool true -> acc+1
            | Bool false -> acc
            | _ -> failwith "type error"
          )
        in Int (List.fold_left check 0 lst)
      | _ -> failwith "type error"
    )
  )
```